

03 Matrix Multiplication

3.3 The Third View of Matrix Multiplication

Linear Combinations of Column Vectors, Column-by-Column Computation, and Heatmap Visualization



Linear Algebra Made Easy and Visual | Open-source code & drafts: <https://github.com/Visualize-ML>

Core Skills for This Section

1

The linear-combination-of-columns view: for matrix multiplication AB , every column of the result is a weighted sum of A 's column vectors

2

Ab is exactly A 's column vectors combined and weighted by the coefficients in b

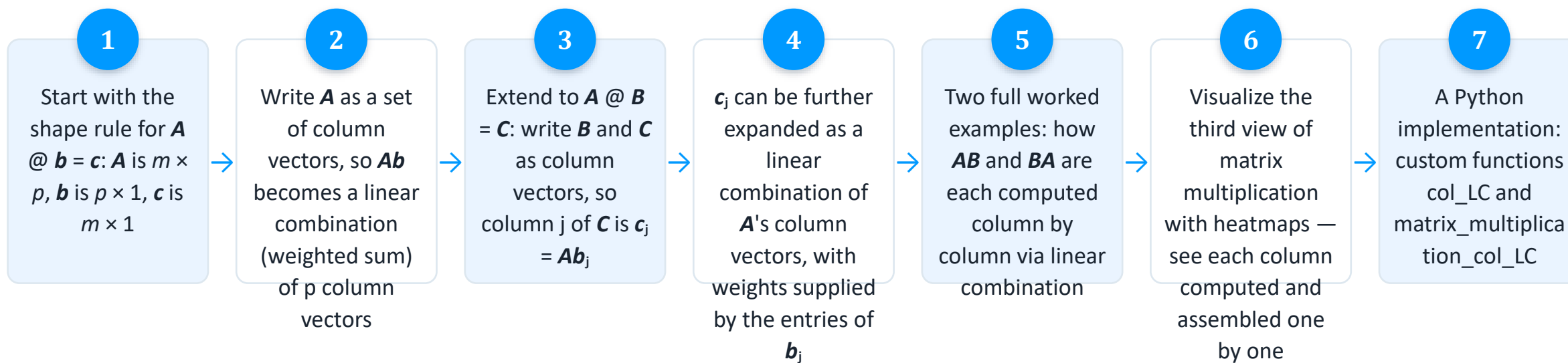
3

For AB , each column of B acts as a set of weights applied to A 's columns, and the weighted sum produces the corresponding column of C

4

For AB , however many columns B has, AB has that many columns too

How This Section Unfolds



Seeing Matrix Multiplication as a Linear Combination of Columns

The third view of matrix multiplication writes the left-hand matrix as a set of column vectors — every column of the result is a linear combination of that set

- The previous two sections showed matrix multiplication can be “expanded by inner products” or “expanded by outer products”; this section shows it can equally be understood through linear combinations of column vectors
- This view starts from the columns and reveals yet another essence of matrix multiplication: every column of $\mathbf{C}$ is the result of a linear combination of $\mathbf{A}$'s columns

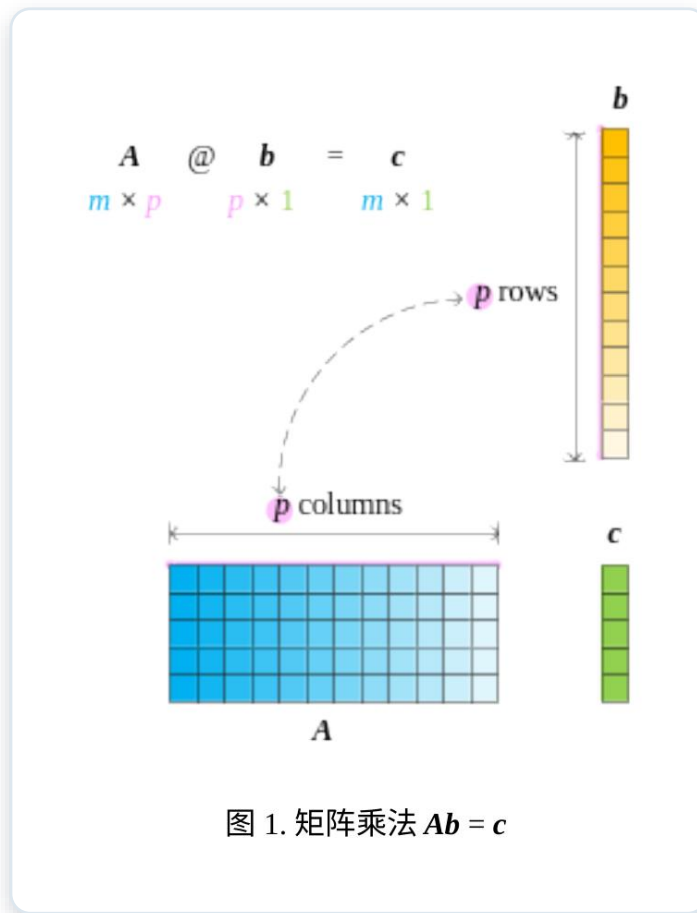
First, $\mathbf{A} @ \mathbf{b} = \mathbf{c}$

Matrix $\mathbf{A}$ has shape $m \times p$, column vector $\mathbf{b}$ has shape $p \times 1$; their product $\mathbf{A}\mathbf{b} = \mathbf{c}$ has shape $m \times 1$

- This is the most basic case of a matrix multiplying a column vector, and also the starting point for understanding the third view
- Next, write $\mathbf{A}$ as a set of column vectors and see exactly how $\mathbf{c}$ gets computed

Figure 1 · Matrix Multiplication $Ab = c$

- A is an $m \times p$ matrix, b is a $p \times 1$ column vector; A 's p columns line up with b 's p rows, and the shared dimension p “cancels out”
- The result c has shape $m \times 1$, matching A 's row count m



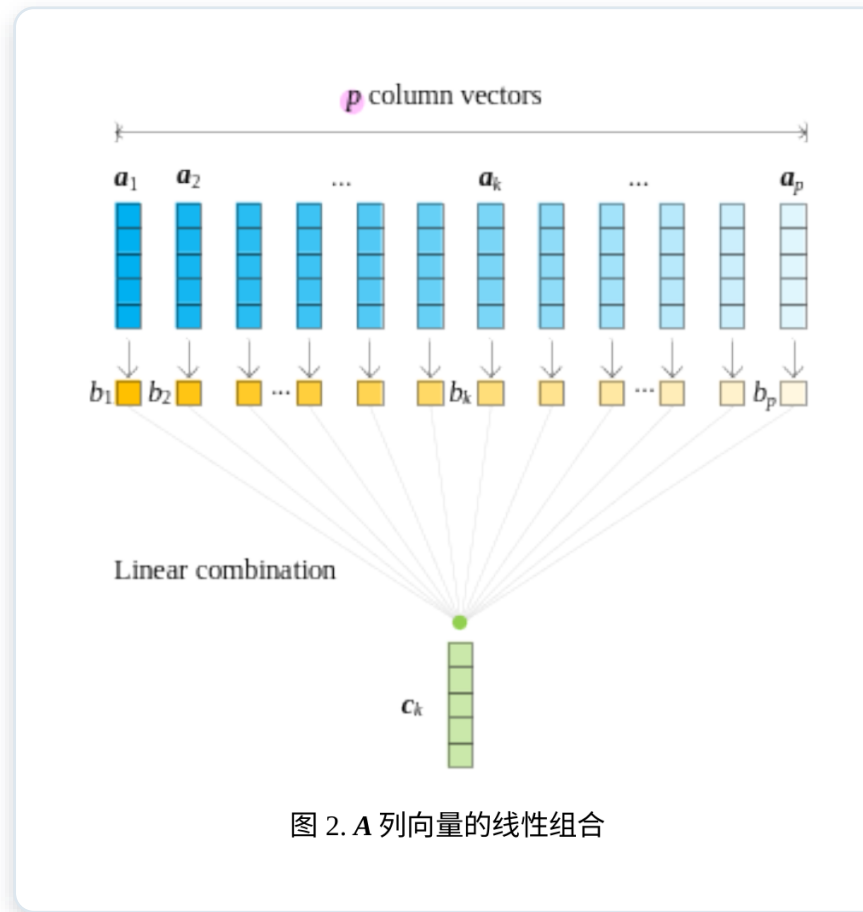
Writing $\mathbf{A}$ as a Set of Column Vectors

$$\mathbf{A}_{m \times p} \mathbf{b}_{p \times 1} = [\mathbf{a}_1 \ \mathbf{a}_2 \ \cdots \ \mathbf{a}_p] [b_1; b_2; \vdots; b_p] = b_1 \mathbf{a}_1 + b_2 \mathbf{a}_2 + \cdots + b_p \mathbf{a}_p = \mathbf{c}$$

- Column vector $\mathbf{a}_k$ ($k = 1, 2, \dots, p$) has shape $m \times 1$; matrix multiplication $\mathbf{A} @ \mathbf{b}$ can be written as the equation above
- $\mathbf{c}$ is a linear combination (weighted sum) of $\mathbf{A}$'s p column vectors — this is the third view of matrix multiplication, the linear-combination-of-columns view

Figure 2 · A Linear Combination of A's Column Vectors

- A 's p column vectors $\mathbf{a}_1, \mathbf{a}_2, \dots, \mathbf{a}_p$ are each multiplied by the corresponding coefficient $b_1, b_2, \dots, b_p$ from $\mathbf{b}$
- Adding these p weighted column vectors together produces the result vector $\mathbf{c}_k$ — that is exactly what “linear combination” means



Now $A @ B = C$

$$C = [c_1 \ c_2 \ \cdots c_n] = A @ B = A @ [b_1 \ b_2 \ \cdots b_n] = [Ab_1 \ Ab_2 \ \cdots Ab_n]$$

- The linear-combination-of-columns view extends naturally to $C = A @ B$: write both C and B as sets of column vectors
- B has n columns, so $A @ B$ also has n columns — B 's column count determines the product's column count

Figure 3 · Writing C and B as Sets of Column Vectors

- Each column b_j of B ($j = 1, 2, \dots, n$) corresponds to the column c_j in the same position of C
- A is $m \times p$, B is $p \times n$; the shared dimension p “cancels out,” leaving C with shape $m \times n$

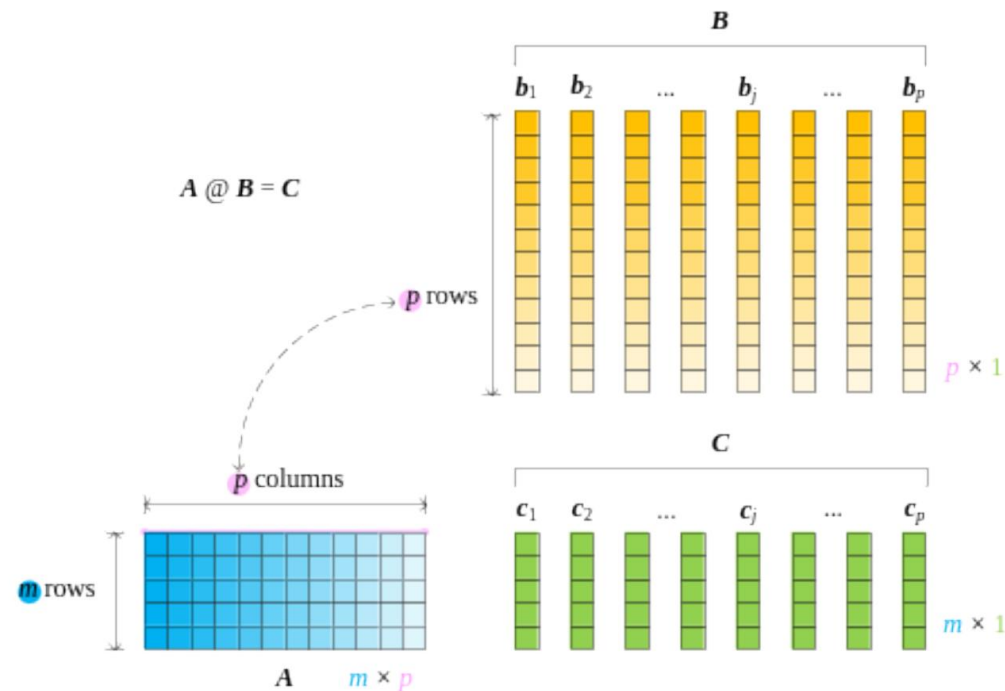


图 3. 把矩阵 C 、 B 分别写成一组列向量

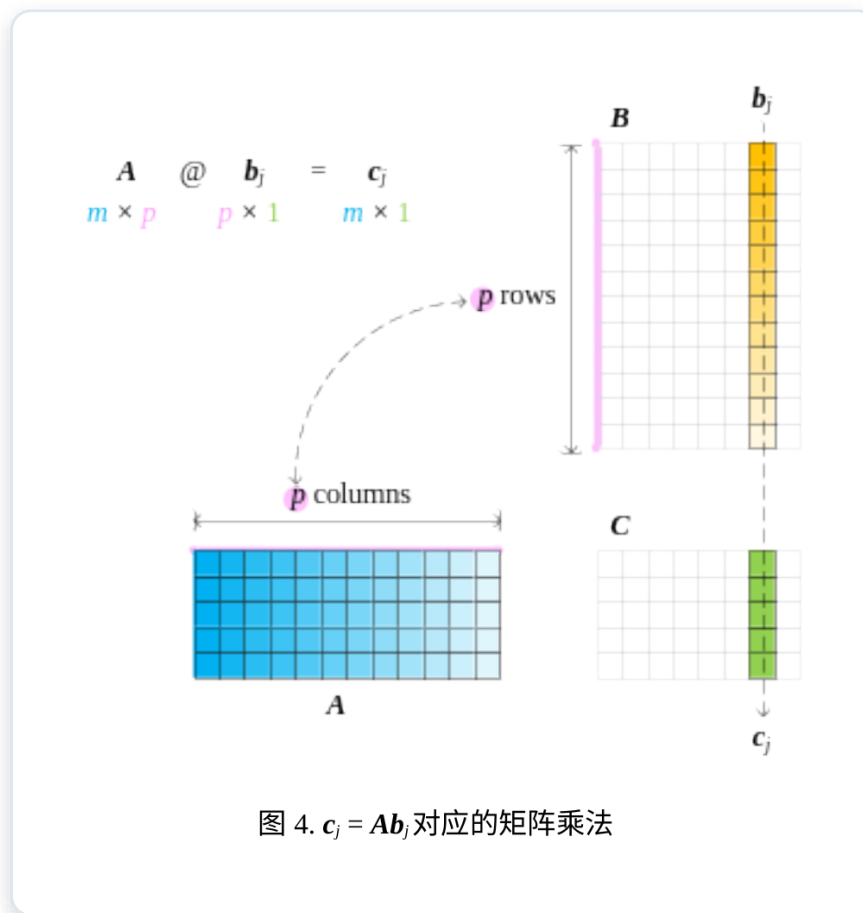
Any Column of $\mathbf{C}$ Can Be Computed Column by Column

$$\mathbf{c}_j = \mathbf{A} @ \mathbf{b}_j$$

- Column j of $\mathbf{C}$ is simply the product of $\mathbf{A}$ with column j of $\mathbf{B}$ — this breaks matrix multiplication into n separate “matrix times column vector” operations
- This is exactly the same case as Figure 1's $\mathbf{A}\mathbf{b} = \mathbf{c}$, just applied here to compute one column of $\mathbf{C}$ at a time

Figure 4 · The Matrix Multiplication Behind $c_j = Ab_j$

- A stays fixed; taking column j of B , b_j , and multiplying by A gives exactly column j of C , c_j
- Repeating this for $j = 1$ through n computes every column of C , assembling the complete $C = A @ B$



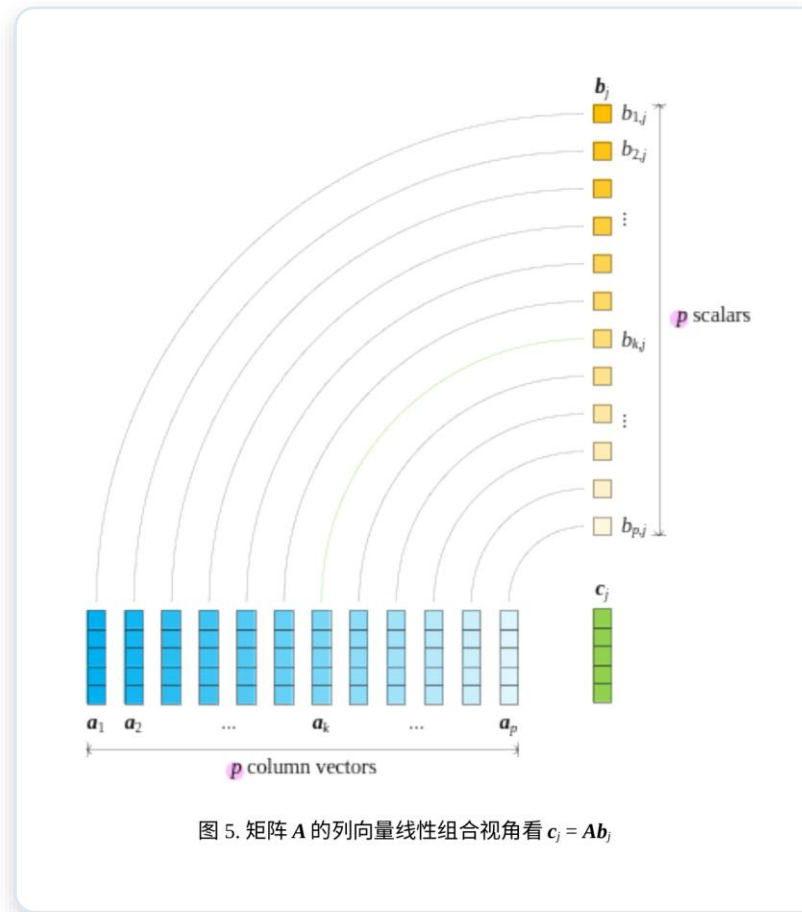
$\mathbf{c}_j$ Can Be Further Written as a Linear Combination of $\mathbf{A}$'s Columns

$$\mathbf{c}_j = \mathbf{A} @ \mathbf{b}_j = [\mathbf{a}_1 \ \mathbf{a}_2 \ \cdots \ \mathbf{a}_p][b_{1,j}; b_{2,j}; \vdots; b_{p,j}] = b_{1,j}\mathbf{a}_1 + b_{2,j}\mathbf{a}_2 + \cdots + b_{p,j}\mathbf{a}_p$$

- The combination coefficients come from column j of $\mathbf{B}$, $\mathbf{b}_j$ — its p scalar entries are exactly p weights
- Working backward: after computing $\mathbf{A}\mathbf{b}_1, \mathbf{A}\mathbf{b}_2, \dots$, one by one, lining them up in order produces matrix $\mathbf{C}$

Figure 5 · The Linear-Combination View of $\mathbf{c}_j = \mathbf{A}\mathbf{b}_j$

- $\mathbf{b}_j$'s p entries $b_{1,j}, b_{2,j}, \dots, b_{p,j}$ are each the weight for one of $\mathbf{A}$'s p column vectors $\mathbf{a}_1, \mathbf{a}_2, \dots, \mathbf{a}_p$
- Summing the weighted columns gives $\mathbf{c}_j$ — this figure unifies “column-by-column computation” and “linear combination of columns” into one picture



Back to an Earlier Example: Symbolically Expand $A @ b_1$ and $A @ b_2$

$$c_1 = A @ b_1 = [a_1 \ a_2 \ a_3][b_{1,1}; b_{2,1}; b_{3,1}] = b_{1,1}a_1 + b_{2,1}a_2 + b_{3,1}a_3$$

- $A = [1,2,3; 4,5,6]$, $B = [1,4; 2,5; 3,6]$; first look at column 1, $b_1 = [1,2,3]^T$
- Similarly, $c_2 = A @ b_2 = b_{1,2}a_1 + b_{2,2}a_2 + b_{3,2}a_3$, where $b_2 = [4,5,6]^T$

Computing $\mathbf{c}_1$ and $\mathbf{c}_2$

$$\mathbf{c}_1 = 1 \times [1, 4]^T + 2 \times [2, 5]^T + 3 \times [3, 6]^T = [14, 32]^T; \mathbf{c}_2 = 4 \times [1, 4]^T + 5 \times [2, 5]^T + 6 \times [3, 6]^T = [32, 77]^T$$

- $\mathbf{a}_1 = [1, 4]^T$, $\mathbf{a}_2 = [2, 5]^T$, $\mathbf{a}_3 = [3, 6]^T$ are $\mathbf{A}$'s three columns, weighted by the entries of $\mathbf{b}_1$ and $\mathbf{b}_2$ respectively
- Weighting each column vector by its coefficient and summing term by term gives $\mathbf{c}_1 = [14, 32]^T$ and $\mathbf{c}_2 = [32, 77]^T$

Figure 6 · Computing c_1 and c_2 via Linear Combination

- Row 1: $1 \times a_1 + 2 \times a_2 + 3 \times a_3 = c_1 = [14, 32]^T$
- Row 2: $4 \times a_1 + 5 \times a_2 + 6 \times a_3 = c_2 = [32, 77]^T$

$$\begin{array}{l} b_{1,1} \times \begin{array}{|c|} \hline a_1 \\ \hline 1 \\ \hline 4 \\ \hline \end{array} + b_{2,1} \times \begin{array}{|c|} \hline a_2 \\ \hline 2 \\ \hline 5 \\ \hline \end{array} + b_{3,1} \times \begin{array}{|c|} \hline a_3 \\ \hline 3 \\ \hline 6 \\ \hline \end{array} = \begin{array}{|c|} \hline c_1 \\ \hline 14 \\ \hline 32 \\ \hline \end{array} \\ \\ b_{1,2} \times \begin{array}{|c|} \hline a_1 \\ \hline 1 \\ \hline 4 \\ \hline \end{array} + b_{2,2} \times \begin{array}{|c|} \hline a_2 \\ \hline 2 \\ \hline 5 \\ \hline \end{array} + b_{3,2} \times \begin{array}{|c|} \hline a_3 \\ \hline 3 \\ \hline 6 \\ \hline \end{array} = \begin{array}{|c|} \hline c_2 \\ \hline 32 \\ \hline 77 \\ \hline \end{array} \end{array}$$

图 6. 通过线性组合计算 c_1 、 c_2

Placing c_1 and c_2 Side by Side Gives Matrix C

$$C = A @ B = [c_1 \ c_2] = [14, 32; 32, 77]$$

- Once c_1 and c_2 are computed, lining them up left to right assembles the complete C
- This matches the results from the first and second views exactly — different paths, same destination

Figure 7 · Placing c_1 and c_2 Side by Side to Form Matrix C

- $c_1 = [14, 32]^T$ comes from the linear combination of A with $b_1 = [1, 2, 3]^T$; $c_2 = [32, 77]^T$ comes from A with $b_2 = [4, 5, 6]^T$
- Placing c_1 and c_2 side by side gives $C = [14, 32; 32, 77]$

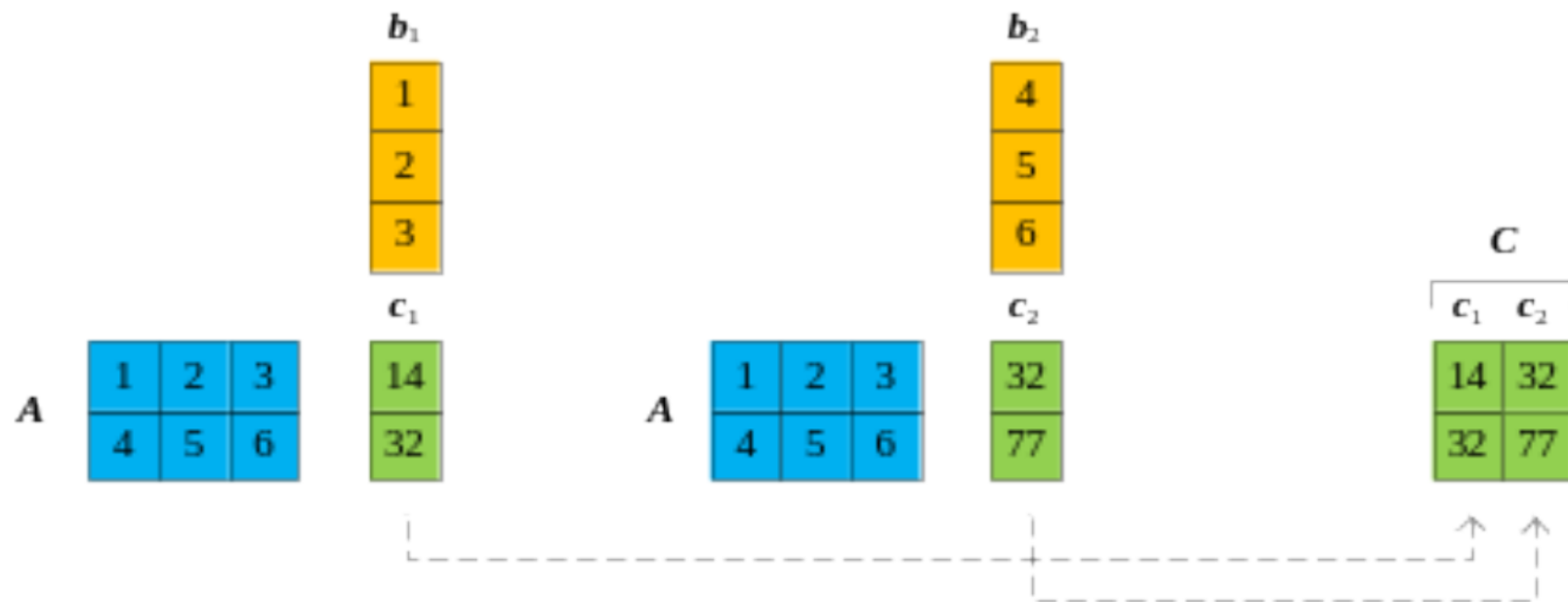


图 7. 把列向量 c_1 、 c_2 顺序摆放得到矩阵 C

The Linear-Combination View of the Second Example: $B @ A$

$$\mathbf{d}_1 = \mathbf{B} @ \mathbf{a}_1 = 1 \times \mathbf{b}_1 + 4 \times \mathbf{b}_2 = [17, 22, 27]^T; \mathbf{d}_2 = 2 \times \mathbf{b}_1 + 5 \times \mathbf{b}_2 = [22, 29, 36]^T; \mathbf{d}_3 = 3 \times \mathbf{b}_1 + 6 \times \mathbf{b}_2 = [27, 36, 45]^T$$

- Here the roles switch: the left-hand matrix is $\mathbf{B} = [1, 4; 2, 5; 3, 6]$, and the coefficients come from $\mathbf{A} = [1, 2, 3; 4, 5, 6]$'s columns $\mathbf{a}_1 = [1, 4]^T$, $\mathbf{a}_2 = [2, 5]^T$, $\mathbf{a}_3 = [3, 6]^T$
- $\mathbf{B}$ has only 2 columns, so every $\mathbf{d}_j$ is a linear combination of the two columns $\mathbf{b}_1$ and $\mathbf{b}_2$

Figure 8 · Computing d_1 , d_2 , and d_3 via Linear Combination

- Row 1: $1 \times b_1 + 4 \times b_2 = d_1 = [17, 22, 27]^T$
- Rows 2 and 3 give $d_2 = [22, 29, 36]^T$ and $d_3 = [27, 36, 45]^T$, with weights taken from A 's 2nd and 3rd columns

Figure 8 illustrates the calculation of d_1 , d_2 , and d_3 via linear combinations of b_1 and b_2 .

Row 1: $a_{1,1} \times b_1 + a_{2,1} \times b_2 = d_1$

	b_1		b_2		d_1	
$a_{1,1}$	$\begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$	$\times$	$a_{2,1}$	$\begin{bmatrix} 4 \\ 5 \\ 6 \end{bmatrix}$	$=$	$\begin{bmatrix} 17 \\ 22 \\ 27 \end{bmatrix}$
1			4			

Row 2: $a_{1,2} \times b_1 + a_{2,2} \times b_2 = d_2$

	b_1		b_2		d_2	
$a_{1,2}$	$\begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$	$\times$	$a_{2,2}$	$\begin{bmatrix} 4 \\ 5 \\ 6 \end{bmatrix}$	$=$	$\begin{bmatrix} 22 \\ 29 \\ 36 \end{bmatrix}$
2			5			

Row 3: $a_{1,3} \times b_1 + a_{2,3} \times b_2 = d_3$

	b_1		b_2		d_3	
$a_{1,3}$	$\begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$	$\times$	$a_{2,3}$	$\begin{bmatrix} 4 \\ 5 \\ 6 \end{bmatrix}$	$=$	$\begin{bmatrix} 27 \\ 36 \\ 45 \end{bmatrix}$
3			6			

图 8. 通过线性组合计算 d_1 、 d_2 、 d_3

Placing D1, D2, and D3 Side by Side Gives Matrix D

$$\mathbf{D} = \mathbf{B} @ \mathbf{A} = [\mathbf{d}_1 \ \mathbf{d}_2 \ \mathbf{d}_3] = [17,22,27; 22,29,36; 27,36,45]$$

- $\mathbf{A}$ has 3 columns, so $\mathbf{D} = \mathbf{B} @ \mathbf{A}$ needs three linear combinations — matching $\mathbf{D}$'s three columns
- This result matches what the previous two sections computed for $\mathbf{D}$, confirming once again that all three views arrive at the same destination

Figure 9 · Placing D_1 , D_2 , and D_3 Side by Side to Form Matrix D

- d_1, d_2, d_3 each get their weights from A 's 1st, 2nd, and 3rd columns, applied to B 's two column vectors b_1 and b_2
- Placing the three columns in order assembles $D = [17, 22, 27; 22, 29, 36; 27, 36, 45]$

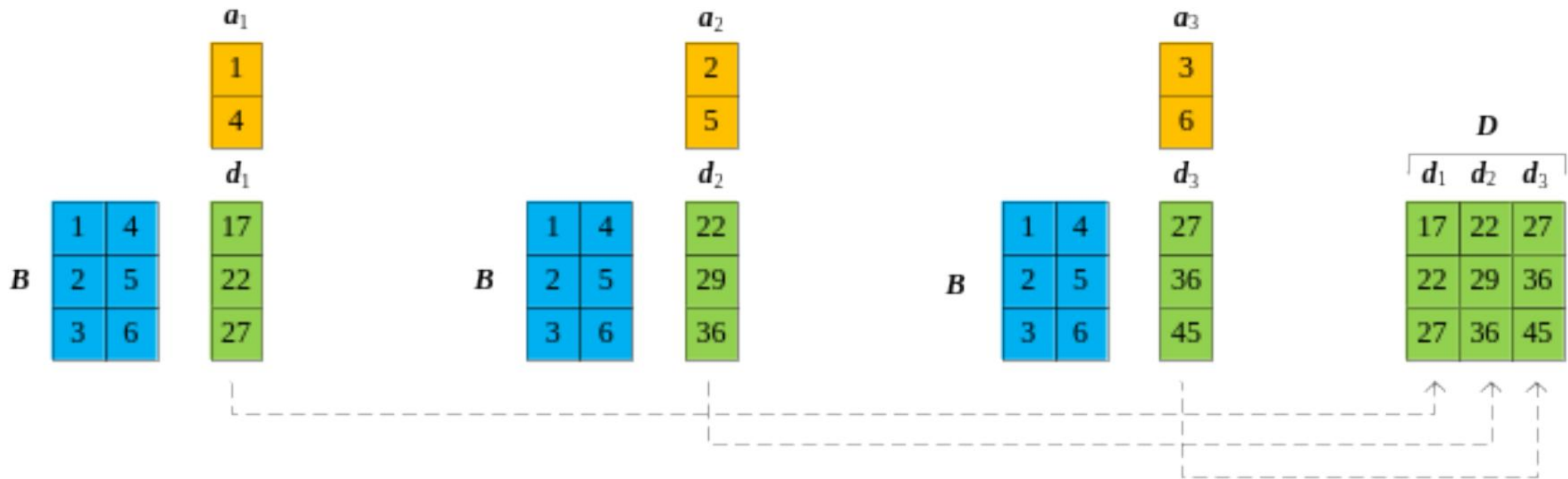


图 9. 把列向量 d_1 、 d_2 、 d_3 顺序摆放得到矩阵 D

Using Heatmaps to Show the Third View of Matrix Multiplication

Color intensity represents the magnitude of each entry; heatmaps let us “watch” each column $A\mathbf{b}_j$ get computed and assembled one by one into $A @ B$

- Given a pair of matrices A, B where B has 8 columns, first look at the heatmap showing how $A @ \mathbf{b}_1$ is computed
- Then see all 8 of the $A @ \mathbf{b}_j$ products lined up side by side, assembling the complete $C = A @ B$

Figure 10 · Heatmap Showing $A @ b_1$

- b_1 is B 's first column (color represents magnitude), combined with A 's columns via weighted sum to produce the $A @ b_1$ column heatmap
- This is the first step of column-by-column computation; the remaining seven columns follow the exact same process

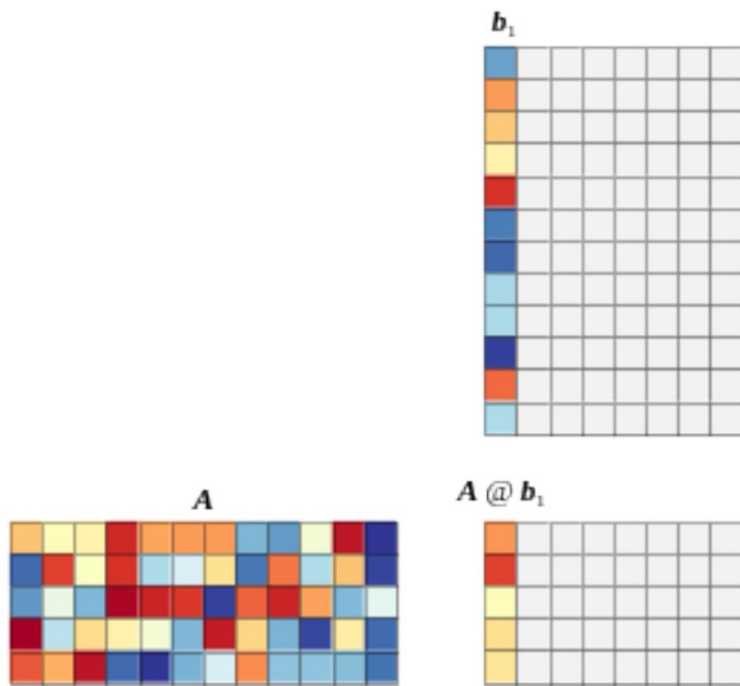


图 10. 热图展示 $A @ b_1$

Figure 11 · The 8 Products $A @ b_j$ Lined Up to Form $C = A @ B$

- B has 8 columns, so there are 8 products altogether, just like $A @ b_1$, corresponding to c_1 through c_8
- Lining up these 8 result columns in order assembles the complete $C = A @ B$

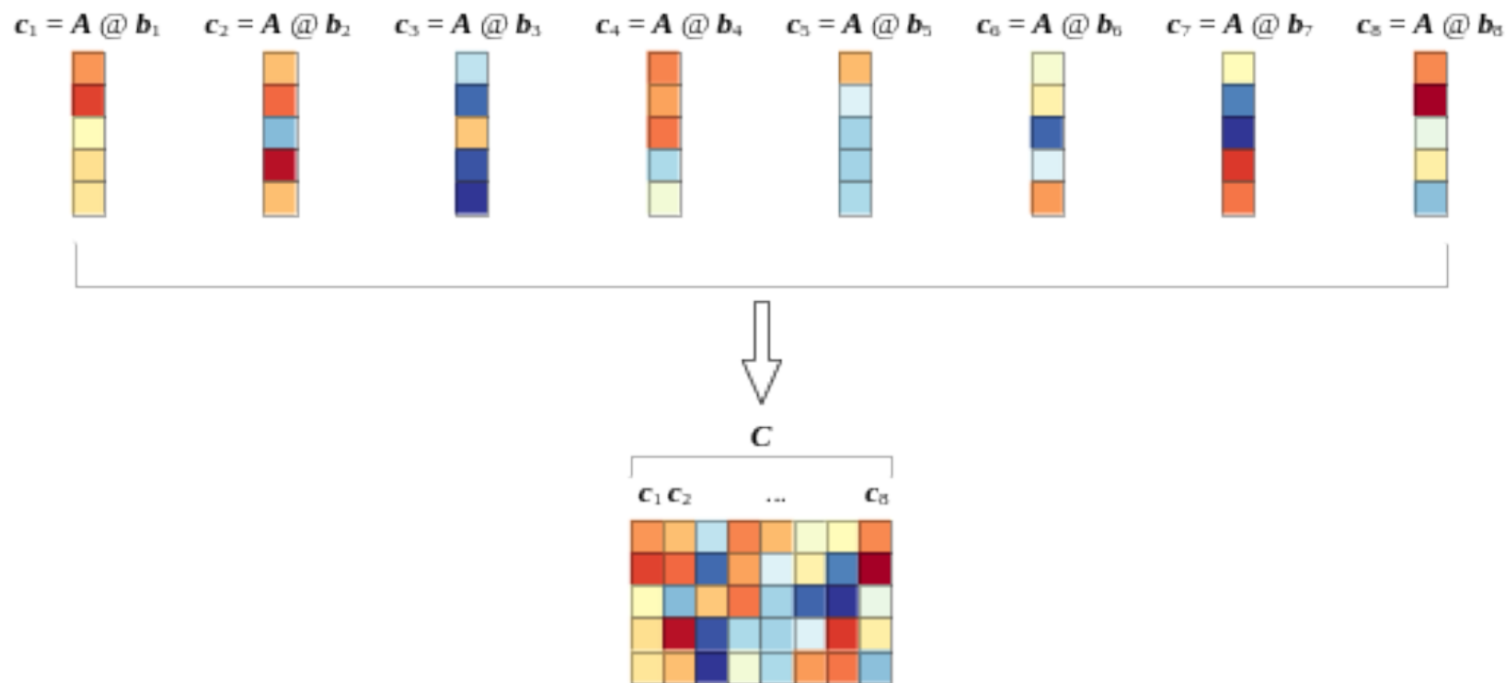


图 11. 8 个 $A @ b_j$ 左右排列得到矩阵 $C = A @ B$

Computing Matrix Multiplication via Linear Combinations of Columns

First define $\text{col_LC}(A, \text{coefficients})$ to compute a linear combination of columns, then define $\text{matrix_multiplication_col_LC}(A, B)$, which calls it column by column to complete the whole multiplication

- col_LC uses a single for-loop over each column of $\mathbf{A}$, multiplies it by the matching coefficient, and accumulates the result into a linear-combination vector
- $\text{matrix_multiplication_col_LC}$ uses a single for-loop over each column of $\mathbf{B}$, passing that column as the coefficients to col_LC , filling result matrix $\mathbf{C}$ one column at a time

Code 1 · A Custom Python Function for Matrix Multiplication

LA_03_03_03.ipynb

- `def col_LC(A, coefficients):` takes each column $A[:, k]$, multiplies by `coefficients[k]`, and accumulates — producing the linear-combination result
- `def matrix_multiplication_col_LC(A, B):` first checks whether p_A equals p_B , raising an error if not, then initializes result matrix C as an (m, n) matrix of zeros
- Looping $j = 0$ to $n-1$, it takes column j of B as the coefficients (`coeffs = B[:, j]`) and calls `C[:, j] = col_LC(A, coeffs)` to fill in C one column at a time

```
## 初始化
import numpy as np

## 定义列向量线性组合函数
def col_LC(A, coefficients):
    m, p = A.shape
    col_LC_result = np.zeros(m) # 初始化组合结果

    for k in range(p):
        col_LC_result += coefficients[k] * A[:, k]
        # A (左侧) 每列乘系数并相加
    return col_LC_result

## 定义矩阵乘法函数, 列向量组合
def matrix_multiplication_col_LC(A, B):
    # 获取矩阵 A 和 B 的形状
    m, p_A = A.shape
    p_B, n = B.shape

    # 检测矩阵形状是否符合矩阵乘法规则
    if p_A != p_B:
        raise ValueError('Dimensions do not match')

    # 初始化结果矩阵 C, 形状 (m, n), 初始值设为 0
    C = np.zeros((m, n))

    for j in range(n):
        coeffs = B[:, j]
        # 取出 B 的第 j 列, 作为线性组合的系数
        C[:, j] = col_LC(A, coeffs)
        # 计算 A 的列的线性组合

    return C

## 矩阵乘法
A = np.array([[1, 2, 3],
              [4, 5, 6]])
B = A.T

## 矩阵乘法
matrix_multiplication_col_LC(A, B)
matrix_multiplication_col_LC(B, A)
```

Linear Combinations of Columns: The Third Key to Matrix Multiplication

Treat each column of B as a set of weights applied to A 's columns, combine and assemble column by column — this is the third key to understanding the structure of matrix multiplication

- Inner-product expansion, outer-product expansion, and linear combination of columns are three different views of the same matrix multiplication AB , and they always agree
- This view lays the groundwork for later topics: column space, solutions of linear systems, and matrix rank

Key Terms in This Section

Linear Combination of Columns $Ab = b_1a_1 + b_2a_2 + \dots + b_pa_p$

Combination Coefficients The entries $b_{1,j}$, $b_{2,j}$, ... of b_j serve as the weights in the linear combination

Identity Matrix Satisfies $A @ I = A$, using standard basis vectors as coefficients

The col_LC Function A custom Python function computing a linear combination of columns given coefficients

Single For-Loop (Column View) Iterating $j = 1$ to n , computing and filling in one column of C per step

The Third View of Matrix Multiplication Column j of C , c_j , is a linear combination of A 's columns, weighted by column j of B

Column-by-Column Computation $c_j = Ab_j$ — breaking matrix multiplication into n “matrix times column vector” steps

Permutation Matrix Used to reorder a matrix's columns (or rows)

The matrix_multiplication_col_LC Function Calls `col_LC` column by column to complete the full matrix multiplication

Heatmap (Third View) A visualization showing each column Ab_j computed and assembled into $A @ B$

GO FURTHER

Companion Code and Further Reading

Further Reading · LA_03_03_01.ipynb

Computes both worked examples in this section using the third view of matrix multiplication

Further Reading · LA_03_03_02.ipynb

Uses `seaborn.heatmap()` to visualize the third view of matrix multiplication

Further Reading · LA_03_03_03.ipynb

A custom Python function for matrix multiplication, implemented from the linear-combination-of-columns view

Open-Source Repository

Code and figures for the whole book: <https://github.com/Visualize-ML>

Questions Q1 – Q10

Q1 Use the linear-combination-of-columns view to compute Ab for the following pairs: ① $A=[1,2;3,4]$, $b=[1,2]^T$; ② $A=[1,2;3,4;5,6]$, $b=[1,2]^T$; ③ $A=[1,2,3;4,5,6]$, $b=[1,2,3]^T$; ④ $A=[1,2,3;4,5,6;7,8,9]$, $b=[1,2,3]^T$.

Q3 Combine the following two matrix multiplications into a single one: $[1,2;3,4]@[1;1]$ and $[1,2;3,4]@[-1;1]$.

Q5 Combine the following three matrix multiplications into a single one, and give the result: $[1,2,3;4,5,6]@[0;0;1]$, $@[0;1;0]$, $@[1;0;0]$.

Q7 Study permutation matrices on your own.

Q9 Write a function `col_lc_matmul(A, b)` that computes $A @ b$ using the linear-combination-of-columns approach, and compare it against `np.dot(A, b)`.

Q2 Use the third view of matrix multiplication to expand AB for the following pairs: ① $A=[1,2;3,4]$, $B=[1,-1;1,1]$; ② $A=[1,2;3,4;5,6]$, $B=[1,-1;1,1]$; ③ $A=[1,2,3;4,5,6]$, $B=A^T$; ④ $A=[1,2,3;4,5,6;7,8,9]$, $B=A^T$.

Q4 Combine the following three matrix multiplications into a single one, and give the result: $[1,2,3;4,5,6]@[1;0;0]$, $@[0;1;0]$, $@[0;0;1]$.

Q6 Compare the results of Q4 and Q5, and explain their similarities and differences.

Q8 A chicken has one head and two feet; a rabbit has one head and four feet. A cage currently holds 5 chickens and 8 rabbits. Use matrix multiplication to compute the total number of heads and feet in the cage.

Q10 Use Python to verify the Ab computations for the four pairs in Q1, printing each one's linear-combination expansion (e.g., “ $1 \times a_1 + 2 \times a_2$ ”).

Questions Q11 – Q19

Q11 Modify the `matrix_multiplication_col_LC` function from Code 1, constructing at least 3 shape-mismatched A, B pairs, and verify each correctly raises a `ValueError`.

Q13 Compute the same pair of random matrices' product using three custom functions — the first view (inner product), second view (outer product), and third view (linear combination of columns) — and verify all three results agree exactly.

Q15 For the chicken-and-rabbit problem in Q8, write the matrix multiplication in the third-view linear-combination form, showing step by step how the total heads and feet arise from a linear combination of the “chicken head/feet vector” and the “rabbit head/feet vector.”

Q17 Construct a permutation matrix P (swap two columns of the identity matrix), compute $A @ P$, and observe how the result connects to the patterns in Q4 and Q5.

Q19 Use Python to verify this section's first example: given $A=[1,2,3;4,5,6]$, $B=[1,4;2,5;3,6]$, compute $A @ B$ using the linear-combination-of-columns view, and compare against the first and second views.

Q12 Use the `time` module to compare the runtime of `matrix_multiplication_col_LC` against NumPy's built-in $A @ B$ on large matrices (e.g., 300×300).

Q14 Write a function `decompose_by_columns(A, B)` that decomposes $A @ B$ into n column computations Ab_j , and use matplotlib subplots to show each column's computation step by step.

Q16 Construct a 4×4 identity matrix I , verify whether $A @ I$ equals A itself, and explain why from the linear-combination-of-columns perspective.

Q18 Write a function `random_linear_combination(A)` that randomly generates a coefficient vector b , computes Ab , and plots the column vectors ak and the result c as arrows in a 2D plane (assuming A is a $2 \times p$ matrix).

Questions Q20 – Q28

Q20 Use Python to verify this section's second example: given $B=[1,4;2,5;3,6]$, $A=[1,2,3;4,5,6]$, compute $B @ A$ using the linear-combination-of-columns view, and compare against the first and second views.

Q22 Write a function `check_zero_column(A, b)` that verifies whether Ab is always the zero vector when b is the zero vector, and explain why from the linear-combination perspective.

Q24 Write a function `compare_views(A, B)` that computes $A @ B$ using the first, second, and third views, tallying each implementation's line count and runtime, and summarizing the trade-offs of each view.

Q26 Write a function `matrix_from_basis(A, coeffs_list)` that takes matrix A and a list of coefficient vectors, and returns the new matrix formed by combining A 's columns according to each coefficient vector.

Q28 Combining what you learned in this section, write a small program that, given any matrices A and B , checks whether B 's row count equals A 's column count, and if so, prints the full linear-combination expansion of every column c_j (e.g., " $c_1 = 1 \times a_1 + 4 \times a_2$ ").

Q21 Following Figures 10 and 11, draw heatmaps for a random pair of 5×4 and 4×6 matrices, showing the third view's column-by-column computation and assembly.

Q23 Explore: if some column b_j of matrix B is entirely zero, what is column j of $C = A @ B$? Verify your conjecture with Python.

Q25 Given a 3×3 matrix A , construct three different permutation matrices, multiply each by A , observe how the columns of the result get reordered, and summarize the pattern of what a permutation matrix does.

Q27 Extend the chicken-and-rabbit problem from Q8 to three kinds of animals (chickens, rabbits, and spiders, with $1/2/1$ heads and $2/4/8$ feet respectively), construct the corresponding matrices, and use Python to compute the total heads and feet for a given count of each.

10 Prompts to Explore Further (1/2)

1

Explain why matrix multiplication $A @ b$ can be understood as a linear combination of A 's column vectors, and where the combination coefficients come from.

2

Explain why column j of $A @ B$, c_j , equals $A @ b_j$, and how this connects to the first and second views.

3

Explain the relationship among the three views of matrix multiplication (inner-product expansion, outer-product expansion, linear combination of columns) — do they always produce the same result, and why?

4

Explain why the identity matrix I satisfies $A @ I = A$, from the perspective of linear combinations of columns.

5

Explain what a permutation matrix does, and why $A @ P$ reorders A 's columns (or rows).

10 Prompts to Explore Further (2/2)

6

Explain why the chicken-and-rabbit problem can be solved using the linear-combination-of-columns view of matrix multiplication, and what each matrix and vector represents in practice.

7

Explain why, in the heatmap visualization, computing each column Ab_j and assembling them one by one demonstrates the third view of matrix multiplication.

8

Explain the relationship between linear combinations of columns and the linear-algebra concept of “column space.”

9

Give an example of how the linear-combination-of-columns view is used in practice in machine learning or data science, such as weighted combinations of features.

10

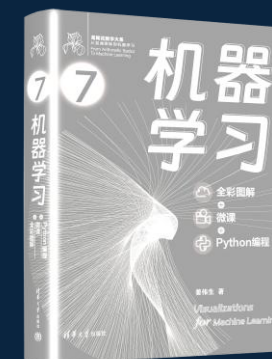
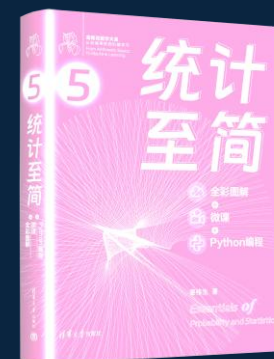
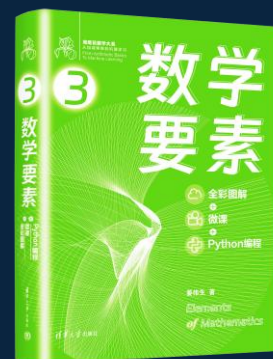
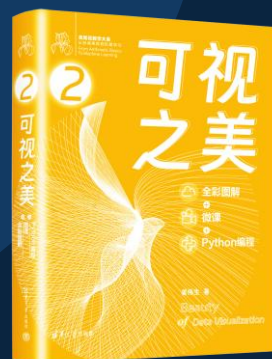
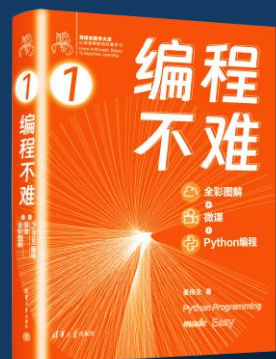
Explain why understanding the linear-combination-of-columns view of matrix multiplication is essential for later topics such as matrix rank, linear independence, and systems of linear equations.

数学

数学基础

线性代数

概率统计



Python编程

数据可视化

工具

数据分析
图和网络

回归、降维
分类、聚类

实践

